

Your First Neural Network

A Beginner's Companion Guide

Classifying a flower-shaped dataset with a single hidden layer — the code, the concepts, and why they matter

© 2026 David Grunwald. All rights reserved.

ABOUT THIS GUIDE

From a Straight Line That Fails to a Curved Boundary That Works

A plain-language walkthrough of a one-hidden-layer neural network

In this project you build a complete neural network — by hand, with NumPy — that learns to separate a “flower”-shaped pattern of red and blue dots. This requires another library – matplotlib – which is not used here for purposes of simplification. It is your first network with a **hidden layer**, and it succeeds at a task where a simpler method fails badly. This guide explains every key idea in plain terms, in the order you meet it, and shows why each one matters in the wider world of deep learning.

The accompanying file, `planar_classification.py`, is fully self-contained and runnable. Everything below refers to the functions inside it. There is a file attached (`code_explained.txt`) that explains the output.

Prerequisites: What You Need to Run the Code

The program is deliberately simple to run. You need:

- **Python 3.8 or newer** installed on your computer.
- **Two libraries:** `numpy` (for all the math) and `scikit-learn` (used only for the logistic-regression comparison). Install both with one command inside your virtual environment:

```
pip install numpy scikit-learn
```

That is all. Unlike the original course version, this file does *not* require any course-only helper files (such as `planar_utils` or `testCases`) — it generates the flower dataset itself. To run it, open a terminal and type:

```
python planar_classification.py
```

It will print the data shapes, the logistic-regression accuracy, the falling training cost, the neural network’s accuracy, and a small experiment showing how the hidden-layer size affects results.

The Libraries, and How They Map to the Code

Only three imports do real work. The table shows what each is for and where it appears in the program, for easy reference.

Library / import	What it is	Where and how it is used in the code
<code>numpy</code> (<code>import numpy as np</code>)	The core library for numerical computing — arrays and fast math on them.	Everywhere. <code>np.dot</code> for matrix multiplication in forward and backward propagation, <code>np.tanh</code> and <code>np.exp</code> for activations, <code>np.sum</code> , <code>np.log</code> , <code>np.random.randn</code> for initializing weights, and <code>np.zeros</code> for biases.

Library / import	What it is	Where and how it is used in the code
copy (import copy)	A built-in Python module for safely duplicating data.	In update_parameters: copy.deepcopy makes an independent copy of the weights so the update does not accidentally modify the original dictionary.
scikit-learn (sklearn.linear_model)	A general machine-learning toolkit. Used here only for one comparison.	In the demo section: LogisticRegressionCV trains the straight-line baseline so you can see how poorly it does, before the neural network shows it up.

Concept 1: What “Shape” Is, and Why It Matters

Every array of numbers has a **shape** — the list of its dimensions. In this project, the data x has shape $(2, 400)$ and the labels y have shape $(1, 400)$. Read those left to right: X has 2 rows (the two features of each point, its x and y position) and 400 columns (the 400 examples). Y has 1 row (a single yes/no label) and 400 columns.

KEY CONCEPT: SHAPE

A **shape** tells you exactly how many numbers an array holds and how they are arranged. The convention used throughout is *features as rows, examples as columns*, so the first number is “how big is one example” and the second is “how many examples.” You read either one with `.shape`: `X.shape[0]` is the feature count, `X.shape[1]` is the number of examples.

Shapes matter because almost every operation in a network is a matrix multiplication, and matrix multiplication only works when the dimensions line up. If a weight matrix is the wrong shape, the program either crashes immediately or — worse — silently produces nonsense. The assignment’s own motto says it best: keep your dimensions straight and you eliminate a huge share of bugs before they happen.

WHY IT MATTERS IN DEEP LEARNING

Foundation: getting shapes right is the difference between code that runs and code that throws a dimension error. **Advanced:** in large networks with dozens of layers, shape discipline is the single most common source of bugs — a transposed matrix or a misplaced axis can be hard to spot. Building the habit now, on a tiny 2-by-400 array, pays off on networks with millions of values.

Concept 2: Logistic Regression, and How It Does Here

Before building the network, the program tries a simpler, classic method: **logistic regression**. You can think of it as a neural network with no hidden layer at all — it draws a single *straight line* and calls everything on one side “red” and everything on the other side “blue.”

KEY CONCEPT: LOGISTIC REGRESSION

Logistic regression is a method for two-class prediction that separates the classes with a single straight boundary (a line in 2D, a flat plane in higher dimensions). It is fast, simple, and works well — but only when the data can actually be split by a straight line.

On the flower data, it manages only about **47% accuracy** — barely better than flipping a coin. The reason is simple and important: the flower’s petals curl around each other, so *no straight line* can separate red from blue. The data is “not linearly separable.” This failure is the whole motivation for what comes next.

WHY IT MATTERS IN DEEP LEARNING

Foundation: logistic regression is the simplest classifier and a useful baseline — always worth trying first to see whether a problem even needs something fancier. **Advanced:** its limitation (straight lines only) is exactly what nonlinear, multi-layer networks were invented to overcome. Seeing it fail here makes the value of a hidden layer concrete rather than abstract.

Concept 3: Why a Hidden Layer, and What One Is

The fix for the flower problem is to add a **hidden layer** — a layer of neurons that sits between the input and the output. The network in this project has one hidden layer of four neurons.

KEY CONCEPT: THE HIDDEN LAYER

A **hidden layer** is a middle layer of neurons whose values you never directly see (hence “hidden”). Each neuron takes the inputs, computes a weighted sum, and passes it through a *nonlinear* activation function (here, tanh). Stacking a hidden layer between input and output lets the network bend and combine the inputs into shapes a straight line never could.

The magic word is **nonlinear**. The hidden layer’s tanh activation is what lets the network draw a *curved* boundary instead of a straight one. With just four hidden neurons, the network wraps a curve around the flower’s petals and reaches about **90% accuracy** — nearly double the straight-line result. The network is never told what each hidden neuron should detect; it works that out for itself during training.

WHY IT MATTERS IN DEEP LEARNING

Foundation: the hidden layer is what makes a “neural network” more powerful than logistic regression — it is the source of the network’s ability to learn complex patterns. **Advanced:** “deep” learning simply means many hidden layers stacked together, each building on the features of the one before. The single hidden layer here is the seed of every deep architecture that follows.

Concept 4: Network Structure — What It Is and How It Is Defined

Before a network can learn anything, you must decide its **structure**: how many neurons sit in each layer. The function `layer_sizes` does this, returning three numbers — `n_x` (input size), `n_h` (hidden size), and `n_y` (output size).

```
def layer_sizes(X, Y):
    n_x = X.shape[0]    # input features - read from the data
    n_h = 4              # hidden neurons - a choice we make
    n_y = Y.shape[0]    # outputs        - read from the data
    return (n_x, n_h, n_y)
```

KEY CONCEPT: STRUCTURE (LAYER SIZES)

A network's **structure** is the number of neurons in each layer. The input and output sizes are fixed by the problem — `n_x` must match the number of features, and `n_y` the number of outputs. The hidden size `n_h` is a *choice* you make, called a *hyperparameter*.

Structure matters for two reasons. First, the layer sizes determine the exact shapes of every weight matrix and bias, so getting them right is what makes the math line up. Second, the hidden size controls the network's capacity — how complex a pattern it can learn.

CHOOSING THE HIDDEN SIZE: UNDERFIT vs. OVERFIT

`n_x` and `n_y` are dictated by your data — pick them wrong and you get an immediate shape error, so they are self-checking. But `n_h` has no single correct value; you find a good one by experiment. **Too small** and the network *underfits* — it cannot capture the pattern and does poorly everywhere. **Too large** and it tends to *overfit* — great on training data but worse on new data, because it memorizes instead of generalizing. The bonus experiment in the code shows this directly: a hidden size of 1 or 2 reaches only about 67%, while 4 or 5 jumps to about 90%, and growing all the way to 50 brings no further gain. You judge the right size by results, not by reasoning in advance.

Concept 5: Forward and Backward Propagation

Once the structure is set and the weights are initialized, learning happens through two complementary passes through the network.

Forward propagation — making a prediction

Forward propagation pushes the input through the network, layer by layer, until a prediction comes out. Each layer does the same two-step rhythm: a linear combination (multiply by weights, add bias), then a nonlinear activation.

```
Z1 = np.dot(W1, X) + b1    # linear,    hidden layer
A1 = np.tanh(Z1)           # activate, hidden layer
Z2 = np.dot(W2, A1) + b2   # linear,    output layer
A2 = sigmoid(Z2)           # activate, output -> probability
```

KEY CONCEPT: FORWARD PROPAGATION

Forward propagation is the network making a guess: data flows from input to output, each layer transforming it, ending in a prediction. The hidden layer uses tanh; the output uses sigmoid to produce a probability between 0 and 1.

Backward propagation — learning from the mistake

Backward propagation runs the other way. Starting from the error at the output, it works backward through the layers to figure out how much each weight and bias contributed to the mistake — and therefore which way each should move to do better. These “which way to move” values are called *gradients*.

KEY CONCEPT: BACKWARD PROPAGATION (AND GRADIENTS)

Backward propagation computes the **gradient** for every parameter: the direction and amount each weight should change to reduce the error. It reuses the values saved during the forward pass (the “cache”), which is why forward propagation stores them. Backprop is the engine of learning — without it, the network would have no idea how to improve.

WHY IT MATTERS IN DEEP LEARNING

Foundation: every neural network learns by alternating these two passes — predict forward, then correct backward. The pattern is identical whether the network has one layer or a thousand. **Advanced:** backpropagation is the algorithm behind essentially all of modern deep learning; the entire field of training huge models is, at heart, forward and backward propagation repeated at enormous scale.

Concept 6: What “Cost” Is, and Why It Matters

Between the forward and backward passes sits one crucial number: the **cost**. After the network makes its predictions, `compute_cost` measures how wrong they are, all at once, as a single number.

```
logprobs = np.multiply(np.log(A2), Y) + np.multiply(np.log(1 - A2), 1 - Y)
cost = - np.sum(logprobs) / m
```

KEY CONCEPT: THE COST FUNCTION

The **cost** (here, the *cross-entropy* cost) is a single number summarizing how badly the network is doing across all examples right now. It is small when the network is confident and correct, and large when it is confident and wrong. Training has exactly one goal: make this number as small as possible.

Cost matters because it is the target that learning aims at. Backward propagation computes gradients of the cost — it asks “how should each weight change to make this number go down?” Without a cost, the network would have no definition of “better” and no direction to improve in. The cost is the “downhill” that the whole process rolls toward.

Watching the cost is also how you know learning is working. When you run the program, the cost starts near **0.693** — the value of pure uncertainty, when the untrained network is essentially guessing — and falls steadily as training proceeds. That downward march is the visible signature of a network that is learning.

WHY IT MATTERS IN DEEP LEARNING

Foundation: the cost is the single number that defines success and drives every weight update. A falling cost means the network is improving; a stuck or rising cost is your first warning that something is wrong.

Advanced: choosing the right cost function for a task is one of a practitioner’s most powerful levers — specialized costs drive everything from image generation to language models. They all play the same role you see here: one number that says “this is how wrong we are,” which the network then works to shrink.

Putting It All Together

The function `nn_model` assembles every piece into the complete training loop. It initializes once, then repeats four steps thousands of times:

- **Forward propagation** — make a prediction.
- **Compute cost** — measure how wrong it is.
- **Backward propagation** — find the gradients.
- **Update parameters** — take one small step downhill.

After the loop, `predict` uses the learned weights to classify each point, thresholding the output probability at 0.5. The result: a network that wraps a curved boundary around the flower and reaches about 90% accuracy, where the straight line of logistic regression managed only 47%.

THE ONE IDEA TO CARRY FORWARD

Every neural network — from this little flower classifier to the largest models in the world — learns through the same loop: predict forward, measure the cost, propagate backward, update, repeat. Master that cycle here, on a problem you can see and understand, and you have understood how all of deep learning learns. Everything more advanced is this same loop, scaled up and pointed at harder problems! Awesome job. If you are like me, you'll refer to these concepts over and over again. They aren't intuitive. And remember it took teams of people years to figure these simple concepts and make them work together.